

Programation Orientée Objet en Java

Séance : 04

ISCAE

Dr. Mohamed Ould Djibril

December 13, 2018

Institut Supérieur de Comptabilité et d'Administration
des Entreprises - ISCAE

Objectifs du cours d'aujourd'hui

Les classes et les objets en java

- Définition des méthodes
- Héritage en Java
- Redéfinition des méthodes
- Les méthodes et classes abstraites
- Les interface en Java
- le polymorphisme

Définition des méthodes

Les méthodes

Les opérations (fonctions ou méthodes) d'une classe sont définies par :

- ① un type de retour
- ② un nom
- ③ une liste (éventuellement vide) de paramètres typés en entrée
- ④ une suite d'instructions (un bloc) qui constitue le corps de la méthode

Définition des méthodes

SYNTAXE:

```
1  modificateurs typeRetour nomMethode ([parametres]) ←  
    {  
2      /*  
3      corps de la methode qui peut  
4      contenir l'instruction  
5      */  
6  }
```

Définition des méthodes

Modificateur	Rôle
public	la méthode est accessible aux méthodes des autres classes
private	l'usage de la méthode est réservé aux autres méthodes de la même classe
protected	la méthode ne peut être invoquée que par des méthodes de la classe ou de ses sous classes
final	la méthode ne peut être modifiée (redéfinition lors de l'héritage interdite)
static	la méthode appartient simultanément à tous les objets de la classe.
synchronized	la méthode fait partie d'un thread. Elle barre l'accès à son instance pendant son exécution.
native	le code source de la méthode est écrit dans un autre langage

Définition des méthodes

Le mot clé return

- Si la méthode ne retourne rien, alors on utilise **void** comme typeRetour.
- La valeur retournée par la fonction est spécifiée dans le corps de la méthode par l'instruction de retour:
`return expression;`
ou
`return; // (possible uniquement pour une méthode de type void)`

Signature d'une méthode

La signature d'une méthode est: Le nom de la méthode et l'ensemble des types de ses paramètres.

Définition des méthodes

Remarque sur la syntaxe

Si la méthode retourne un tableau alors les caractères [] peuvent être précisés après le type de retour ou après la liste des paramètres :

```
1  int [] getValeurs() { ... }  
2  int getValeurs() [] { ... }
```

Définition des méthodes

Passage des paramètres

- par valeur pour les types primitifs.
- par valeur des références pour les objets

la référence est passée par valeur (i.e. le paramètre est une copie de la référence), mais le contenu de l'objet référencé peut être modifié par la fonction (car la copie de référence pointe vers le même objet...)

Définition des méthodes

EXEMPLE:

```
1  class ClasseTest {
2      void methode1(int j){
3          j+=12;
4      }
5
6      void methode2() {
7          int j = 3;
8          methode1(j);
9          System.out.println("j=" + j); // j=3
10     }
11 }
```

La méthode main

MAIN:

```
1 public class MonApp1 {  
2     public static void main(String[] args) {  
3         System.out.println(" Bonjour");  
4     }  
5 }
```

Les méthodes de classe

- Ce sont des méthodes qui ont un rôle indépendant d'un objet spécifique. Elles exécutent une action indépendante d'une instance particulière de la classe
- La déclaration d'une méthode de classe se fait à l'aide du mot clé **static**
- une **méthode statique** ne peut pas accéder aux variables **non statiques**
- Pour l'appeler il suffit seulement de connaître le nom de la classe



Généralité

Apporte aux projets :

- Réutilisation
- Lisibilité, maintenabilité
- Extensibilité

Mécanisme

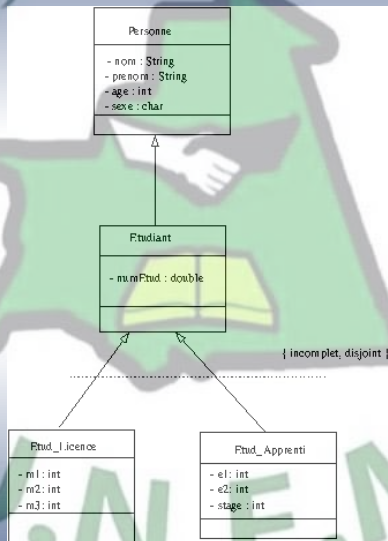
- une instance d'une sous-classe possède la partie structurelle définie dans les superclasses de sa classe plus la partie structurelle définie dans celle-ci
- au niveau du comportement, les objets d'une sous-classe héritent du comportement défini dans les superclasses, avec quelques possibilités de variations, comme nous le verrons plus tard
- au niveau de l'exécution, les langages à objets présentent un mécanisme dénommé liaison (ou ligature) dynamique qui consiste à résoudre l'envoi de message sur les objets (instances), c'est-à-dire à trouver et exécuter le code adéquat correspondant au message



En Java

- toutes les classes dérivent de la classe **Object**, qui est la racine de toute hiérarchie ;
- une classe ne peut avoir qu'une seule super-classe directe ; on parle d'**héritage simple** ;
- Il existe des langages à objets, tels que C++ qui autorise l'**héritage multiple**;
- le mot-clef permettant de définir l'héritage entre classes est **extends**.

Cas d'étude



Structure de la hiérarchie

```
public class Personne{
.....} // fin de la classe Personne
.....
public class Etudiant extends Personne {
.....
} // fin de la classe Etudiant
.....
public class Etud_Licence extends Etudiant {
.....
} // fin de la classe Etud_Licence
.....
public class Etud_Apprenti extends Etudiant {
.....
} // fin de la classe Etud_Apprenti
```


Cas d'étude

- Les attributs définis dans la classe `Personne` sont nom, prénom, âge, sexe. Bien sûr, on définit pour cette classe les constructeurs, les accesseurs, et une opération ou méthode `incrementerAge()` (cette méthode vieillit d'un an).
- Le seul attribut défini dans la classe `Etudiant` est `numEtud`. La classe `Etudiant` définit des opérations de calculs de moyenne, d'admission, de mention.
- Les étudiants de licence (classe `Etud.Licence` passent 3 modules (M1, M2, M3) de coefficient 1. Ils sont admis si leur moyenne générale est ≥ 10
- Les étudiants apprentis (classe `Etud.Apprenti`) passent 2 modules (E1, E2) de coefficient 1 et un stage de coefficient 2. Ils sont admis si leur moyenne est ≥ 10 et si leur note de stage est > 8 .

Cas d'étude

Une instance d'étudiant apprenti (ou une instance d'étudiant licence) a la possibilité d'utiliser des méthodes définies dans *Personne* :

```
// exemple de code pouvant figurer dans la méthode main
// d'une classe application
Etud_Apprenti a = new Etud_Apprenti
    ("Einstein", "Albert", 22, 'M', 123, 8, 8, 10);
// L'age de cet apprenti est 22 ans
a.incrementerAge();
System.out.println
    (a.getName()+"est âgé de " + a.getAge()+" ans");
```

A l'exécution, on obtient :

```
Einstein Albert est âgé de 23 ans
```

Définitions

Signature d'une opération

nom de classe + nom de l'opération + liste des types des paramètres et type de retour

On peut :

- 1 au sein d'une même classe donner le même nom à une opération si la liste des paramètres est différente (**surcharge statique**),
- 2 dans des classes distinctes, donner le même nom et la même liste de paramètres (normal!)
- 3 lorsqu'une méthode d'une sous-classe a même nom et même liste de paramètres qu'une méthode d'une super-classe, on dit que la méthode de la sous-classe **redéfinit** la méthode de la super-classe.

Redéfinition d'une méthode

Rédefinition d'une méthode dans une sous classe

Lorsque l'on code une sous-classe, on a la possibilité de redéfinir une ou plusieurs méthodes de la super-classe, simplement en redéfinissant une méthode qui a le même nom et les mêmes paramètres que celle de la super-classe.

Remarque :

La méthode de la super-classe sera encore accessible mais en la préfixant par le mot-clé **super**, pseudo-variable définie par le système et qui représente l'objet courant vu comme étant du type de la super-classe.

```
// Classe Etudiant
public class Etudiant extends Personne{
.....
public boolean admis()
    {return (moyenne() >= 10;)}
}

// Classe Etud_Apprenti
public class Etud_Apprenti extends Etudiant {
.....
public boolean admis()
{
    return (super.admis() && getnoteSt() > 8);

    //on fait appel ici à l'opération admis() définie
    //dans Etudiant.
    //on aurait pu aussi définir admis dans Etud_Apprenti:
    //return (moyenne() >= 10 && getnoteSt() > 8;)
}
}
```

Rédéfinition des constructeurs

- Dès qu'un constructeur a été défini dans une classe de la hiérarchie de classes, il est nécessaire de définir des constructeurs lui correspondant dans toutes les sous-classes.
- D'autre part, il est commode et intéressant de faire appel aux constructeurs des super-classes que l'on invoque par le mot clé `super` suivi de ses arguments entre parenthèses.
- L'invocation du constructeur de la super-classe doit se faire obligatoirement à la première ligne.

```
// la classe Personne
public class Personne{
    .....
    // constructeur par défaut
    public Personne() {
        nom = ""; prenom = ""; age = 0; sexe = 'F';}
    // autre constructeur
    public Personne(String n, String p, int a, char s)
    {
        nom = n; prenom = p; age = a; sexe = s;
    }
    ....
}
```

```
// la classe Etudiant
public class Etudiant extends Personne{
.....
// constructeur par défaut
public Etudiant(){
    // par défaut appelle super();
    // qui est toujours la première instruction du construc.
    numEtu=-1;
}
// autre constructeur
public Etudiant(String n, String p, int a, char s, double n)
{
    super(n, p, a, s); // appel au 2ième constructeur
                        // de la super-classe Personne;
    numEtu=n;
}
....
}
```



```
// la classe Etu_Apprenti
public class Etu_Apprenti extends Etudiant{

// constructeur par défaut
public Etu_Apprenti ()
{
    // Par défaut, super() est appelée.
    E1=-1; E2=-1; stage=-1;
}

// autre constructeur
public Etu_Apprenti (String n, String p, int a, char s,
                    double n, int e1, int e2, int st)
{
    super(n, p, a, s, n); // appel au 2ième constructeur
                          //d'Etudiant
    E1 = e1; E2 = e2; stage = st;
....
}
```

Classe abstraite

Classe très générale

- qui factorise des attributs et des comportements communs
- dont on ne va pas créer d'instances directes

La classe représentant les étudiants

```
.....  
// la classe est déclarée abstraite  
  
public abstract class Etudiant extends Personne{  
  
}
```

Méthode abstraite

Méthode très générale

- ne possède pas de corps
- indique néanmoins la présence de l'opération

Le calcul de la moyenne d'un étudiant ne peut pas être décrit au niveau de la classe Etudiant

```
1 public abstract class Etudiant extends Personne
2 {
3     .....
4     // la methode est declaree abstraite
5     public abstract double moyenne();
6 }
```

Méthode abstraite $\Rightarrow$ classe abstraite : Une classe qui possède une (ou plusieurs) méthode abstraite est obligatoirement ab

Méthode abstraite

Le calcul de la moyenne d'un étudiant de licence est précisé dans la classe Etud_Licence

```
.....  
public class Etud_Licence extends Etudiant{  
.....  
// on définit la méthode abstraite dans la super-classe  
public double moyenne(){return m1+m2+m3;}  
}
```

Interface en Java

- Une **interface** se limite à présenter des services fournis par une classe
- Un ensemble de :
 - signatures de méthodes publiques (méthodes d'instances abstraites)
 - variables de classes constantes et publiques (variables statiques)

Interface en Java

Améliorer le code

- on décrit des types de manière plus abstraite qu'avec les classes et par conséquent ces types sont plus réutilisables ;
- c'est une technique pour masquer l'implémentation puisqu'on découple la partie publique d'un type de son implémentation ;
- on peut écrire du code plus générique (plus général), au sens où il est décrit sur ces types plus abstraits.

Définition d'une Interface

```
public interface Iquadrilatere
{
    public static final int nbCotes = 4;
    public abstract float perimetre();
}
```

ou (en enlevant les mots-clefs obligatoires)

```
public interface Iquadrilatere
{
    int nbCotes = 4;
    float perimetre();
}
```

Spécialisation d'une interface

```
interface Irectangle extends Iquadrilatere
{
    float angle = 90;
    float angle();
    float largeur();
    float hauteur();
}
```


Implémentation d'une interface / sol.1

```
public class Rectangle implements Irectangle
{
    private float largeur, hauteur;
    public Rectangle(){}
    public Rectangle(float l, float h)
        { largeur=l; hauteur=h; }
    public float perimetre()
        { return 2*largeur()+2*hauteur(); }
    public float angle(){return Irectangle.angle;}
    public float largeur(){return largeur;}
    public float hauteur(){return hauteur;}
}
```

Implémentation d'une interface / sol.2

```
class RectangleTab implements Irectangle
{
    private float tab[]=new float[2];
    public RectangleTab(){}
    public RectangleTab(float l, float h)
        { tab[0]=l; tab[1]=h; }

    public float perimetre()
        { return 2*largeur()+2*hauteur(); }
    public float angle(){return Irectangle.angle;}
    public float largeur(){return tab[0];}
    public float hauteur(){return tab[1];}
}
```

Affectation polymorphe, cast-up

Tout étudiant de licence est un étudiant et a fortiori une personne

```
Personne p;  
Etud_Licence e=new Etud_Licence();  
p=e;
```

Cast-down

Certaines personnes sont des étudiants de licence

```
Personne p=new Etud_Licence();  
Etud_Licence e = (Etud_Licence) p;
```

A éviter au maximum, à contrôler si p ossible

```
Personne p=new Etud_Licence();  
if (p instanceof Etud_Licence)  
Etud_Licence e = (Etud_Licence) p;
```

Appels polymorphes liaison dynamique

```
Vector<Etudiant> listeEtudiants = new Vector<Etudiant>();  
listeEtudiants.add(new Etud_Licence  
    ("Curie", "Marie", 22, 'F', 124, 8, 17, 20));  
listeEtudiants.add(new Etud_Apprenti  
    ("Einstein", "Albert", 22, 'M', 123, 13, 17, 17));  
.....  
// afficher toutes les moyennes  
for (int i=0; i<listeEtudiants.size(); i++)  
{  
    Etudiant etud = listeEtudiants.get(i);  
    // Le polymorphisme a lieu ici:  
    // le calcul de la méthode moyenne() sera  
    // différent si etud est une instance de Etud_Licence  
    // ou de Etud_Apprenti.  
    System.out.println(etud.moyenne());  
}
```

Ce que j'ai appris aujourd'hui

- 1 Comment définir des méthodes dans une classe
- 2 Un exemple d'héritage en Java
- 3 Comment redéfinir des opérations
- 4 Les méthodes et classes abstraites
- 5 Les interface en Java
- 6 un exemple explicitant polymorphisme

Cours suivant

Les aspects syntaxiques du langage java